Práctica: Extracción de campos de PDF

Texto seleccionable, OCR y una interfaz web · Ejecución en CPU

Objetivo

Crear una aplicación que extraiga campos de PDFs con texto seleccionable mediante regex y de PDFs escaneados mediante OCR y regex. Como ampliación adicional, utilizaréis un LLM sobre el texto del OCR. Integraréis la extracción en una interfaz básica con FastAPI, HTML y JavaScript.

El reto es elegir herramientas y modelos que funcionen localmente en CPU, equilibrando resultados, tiempo y memoria. Implementaréis el proyecto desde cero, sin necesidad de GPU ni entrenamiento de modelos.

1. Elegir los documentos y los campos

Las cinco facturas ficticias facilitadas están en español, comparten plantilla y contienen más de 30 campos de cabecera y resumen, además de cinco líneas de productos. Definid un esquema de 12–15 campos de distintos grupos de la tabla, con su significado y formato. Como ampliación, extraed también las líneas de producto en una lista. Podéis descargar otros PDFs de un mismo tipo y formato parecido.

Grupo	Ejemplos de campos disponibles
Factura y fechas	Número de factura, fecha de emisión, vencimiento y moneda.
Pedido y albarán	Número y fecha de pedido, albarán y código de cliente.
Emisor y cliente	Nombre, identificador fiscal, dirección, código postal y ciudad de cada parte.
Contacto	Correo electrónico y teléfono del emisor y del cliente.
Pago y entrega	Forma y condiciones de pago, referencia, IBAN de prueba y dirección de entrega.
Importes	Subtotal bruto, descuento, portes, base imponible, IVA y total.
Líneas de producto	Referencia, descripción, cantidad, precio, descuento, tipo de IVA e importe por línea.

La salida será un JSON con todas las claves. Devolved null si un campo no aparece, es ilegible o resulta ambiguo; cero es un valor distinto. No inventéis ni calculéis información que no esté escrita. Guardad también el texto leído para poder revisar los errores.

2. Descargar cinco PDFs de prueba

Descargad cinco PDFs: tres con texto seleccionable y dos sin texto seleccionable. El material adjunto incluye cinco facturas listas para utilizar, con una misma plantilla y datos diferentes. Utilizadlas tal como se descargan y extraed los campos elegidos. No es necesario crear, rellenar ni modificar documentos.

Parte 1 · PDF con texto seleccionable y regex

3. Leer el texto y localizar los campos

Extraed directamente el texto de cada página mediante una biblioteca como PyMuPDF [1]. Aplicad regex y reglas de contexto para producir el JSON. En esta parte no se necesita OCR. Revisad el texto extraído: su orden puede diferir del aspecto visual del documento.

Consejos sobre regex: buscad valores cerca de etiquetas, contemplad variantes como «Factura» y «N.º factura» y utilizad grupos de captura. Tened en cuenta espacios, saltos de línea, mayúsculas y separadores decimales. Evitad patrones demasiado amplios y no asumáis que el primer número o la primera fecha son los que buscáis [2].

Probad las reglas en varios diseños. Si hay varios candidatos, utilizad el contexto o devolved null. Separad la búsqueda de la normalización: por ejemplo, convertir 1.234,50 en "1234.50" o una fecha conocida al formato acordado.

Parte 2 · PDF escaneado: imagen, OCR y regex

4. Leer los PDFs escaneados con OCR

Implementad la lectura de los PDFs sin texto seleccionable: el programa renderizará cada página como imagen y la enviará al OCR. Aplicad después regex al texto obtenido para extraer los campos elegidos. Reutilizad las reglas de la primera parte y justificad las adaptaciones necesarias.

Consejo: PyMuPDF permite renderizar páginas como PNG o como imágenes en memoria y ajustar la resolución [3]. Empezad con una resolución moderada, por ejemplo 200–300 DPI, y comprobad la legibilidad y el tiempo. Renderizar la página completa conserva etiquetas y distribución; extraer solo las imágenes incrustadas puede perder parte del contenido.

Procesad las páginas en orden y conservad su separación en el texto. Guardad las imágenes utilizadas, la salida original del OCR y el JSON final. Si la lectura falla, revisad idioma, inclinación y contraste antes de cambiar de modelo. No corrijáis manualmente el texto para presentar los resultados del sistema.

5. Elegir un OCR que funcione en CPU

Investigad al menos dos opciones de OCR y probad las viables con los PDFs descargados. Tesseract puede servir como punto de partida [4], pero debéis justificar vuestra elección. Registrad motor, modelos o pesos utilizados, versión, idioma y configuración. Si descartáis una opción, explicad por qué.

Medid calidad de los campos extraídos, tiempo por documento y RAM utilizada en CPU. Indicad también procesador, memoria disponible y tiempo de carga. Un modelo que se ejecuta en CPU puede ser demasiado lento o consumir demasiada memoria para vuestra aplicación; comprobadlo con mediciones.

Parte adicional · OCR seguido de un LLM

6. Extraer campos a partir de la salida del OCR

Como tarea adicional, probad la extracción con un LLM que reciba el mismo texto OCR guardado y devuelva el mismo esquema JSON. Conservad también la solución con regex para comparar. No le enviéis imágenes ni respuestas correctas. Comprobad si mejora la extracción cuando cambian las etiquetas o el orden del texto.

Elegid un modelo pequeño con ejecución local en CPU. Podéis investigar modelos cuantizados y herramientas como llama.cpp [5]. Justificad la elección frente a otra alternativa e indicad nombre, versión o identificador, cuantización, herramienta de ejecución y configuración. Probad su tiempo y consumo de RAM; una API remota no demuestra ejecución local en CPU.

Consejo: el prompt debe definir los campos, pedir JSON y exigir null cuando no haya evidencia suficiente. Guardad el prompt completo, la respuesta original y cualquier reintento. Comprobad tanto el formato como los valores: un JSON válido puede contener datos inventados.

Parte 3 · Interfaz básica con FastAPI

7. Conectar la extracción con una página web

La interfaz debe permitir subir un PDF, elegir texto directo + regex, OCR + regex u OCR + LLM, iniciar el procesamiento y consultar o descargar los campos extraídos. Mostrad el estado de procesamiento y los errores de forma comprensible.

Seguid una estructura sencilla: FastAPI sirve la página y atiende las peticiones HTTP; HTML define los controles, CSS su aspecto y JavaScript envía el archivo y muestra el JSON. La lógica de lectura y extracción debe quedar en módulos Python separados de las rutas de la API.

Estructura y comunicación de la aplicación

```
src/       API FastAPI y módulos de texto, OCR y extracción
frontend/  index.html, app.js y styles.css
pdfs/      los cinco documentos descargados
resultados/ textos leídos, JSON y evidencias
README.md  instalación y ejecución en CPU
```

Flujo básico: navegador → petición HTTP → FastAPI → extracción → respuesta JSON → navegador. FastAPI puede servir index.html desde GET / y los archivos JS/CSS como recursos estáticos [6]. JavaScript envía el PDF mediante fetch y FormData a un POST /api/extract, junto con el método elegido; la API procesa el documento y devuelve los campos [7].

Para esta práctica basta con una petición de procesamiento y su respuesta. Si una tarea tarda mucho, podéis ampliar el flujo con un identificador de trabajo y una ruta GET para consultar el estado. No se exige una base de datos ni un sistema de colas. Verificad desde la interfaz ambos tipos de PDF y un caso de error.

Evaluación de los resultados

Diseñad vuestra propia metodología para evaluar las soluciones desarrolladas. Decidid qué aspectos os interesa estudiar, cómo los vais a valorar y qué pruebas necesitáis. Justificad por qué ese procedimiento es adecuado para vuestra tarea y permite extraer conclusiones sobre los métodos utilizados.

No se exige ninguna métrica concreta ni un formato de presentación determinado. Elegid los criterios y las evidencias que consideréis útiles, explicad cómo los habéis obtenido y aplicad vuestro procedimiento a los cinco PDFs descargados.

Las conclusiones deben apoyarse en los resultados observados. Explicad qué podéis afirmar, qué limitaciones tiene vuestra evaluación y cómo la mejoraríais. Conservad en el repositorio las evidencias necesarias para que otra persona pueda entender y repetir vuestro procedimiento.

Entrega

Entregad el repositorio, los cinco PDFs utilizados y un informe sencillo de 1–2 páginas. El repositorio debe incluir el código Python y de la interfaz, dependencias, configuración y un README que permita instalar y ejecutar el proyecto en CPU desde cero. Indicad cómo obtener los modelos y fijad sus versiones; no es necesario incluir pesos grandes.

Adjuntad los cinco PDFs y los resultados de extracción. El informe resumirá la tarea, los modelos OCR y LLM utilizados, su configuración y el motivo de la elección para CPU. Explicad brevemente vuestra metodología de evaluación, resultados y limitaciones, y responded de forma concisa a las preguntas. Guardad las evidencias detalladas en el repositorio y referenciadlas desde el informe.

Preguntas de reflexión

1. ¿Qué cambia entre extraer texto seleccionable y leer un PDF escaneado?
2. ¿Qué OCR elegisteis para CPU y qué mediciones justifican esa elección?
3. ¿Qué aporta el LLM y compensa su tiempo y consumo de memoria?
4. ¿Qué hacen HTML, JavaScript, HTTP y FastAPI durante una extracción?
5. ¿Qué errores requieren revisión humana y cómo ampliaríais la evaluación?

Documentación de consulta

[1] PyMuPDF: extracción de texto

[2] Python: expresiones regulares (re)

[3] PyMuPDF: convertir páginas en imágenes

[4] Tesseract: manual y modelos

[5] llama.cpp: ejecución y cuantización

[6] FastAPI: archivos estáticos

[7] FastAPI: subida de archivos

Anexo · Cinco facturas con una misma plantilla

Las cinco facturas se han creado para esta práctica. Están en español, tienen una página y comparten diseño y etiquetas, con datos diferentes. Incluyen identificación y contacto de emisor y cliente, pedido, albarán, vencimiento, pago, entrega y resumen de importes. Las tres primeras permiten seleccionar texto; las dos últimas contienen únicamente la imagen de la página.

Descarga del material

Descargad el paquete Material_Practica_OCR.zip que se facilita junto al enunciado. Al descomprimirlo encontraréis este documento y la carpeta facturas_practica con los cinco PDFs. También se facilita un ZIP con solo las cinco facturas. No tenéis que preparar los documentos.

1. Factura 01 · Texto seleccionable

Extraed el texto directamente con PyMuPDF y aplicad regex.

Factura_01_texto.pdf

2. Factura 02 · Texto seleccionable

Extraed el texto directamente con PyMuPDF y aplicad regex.

Factura_02_texto.pdf

3. Factura 03 · Texto seleccionable

Extraed el texto directamente con PyMuPDF y aplicad regex.

Factura_03_texto.pdf

4. Factura 04 · PDF de imagen

Leed el documento con OCR y aplicad regex al texto obtenido. Utilizad también esa salida para la parte adicional con LLM.

Factura_04_imagen.pdf

5. Factura 05 · PDF de imagen

Leed el documento con OCR y aplicad regex al texto obtenido. Utilizad también esa salida para la parte adicional con LLM.

Factura_05_imagen.pdf

Los enlaces anteriores abren los PDFs de la carpeta facturas_practica una vez descomprimido el paquete completo. Todos los datos son ficticios. Cada factura ofrece más de 30 campos y cinco líneas con referencia, descripción, cantidad, precio unitario, descuento, IVA e importe. Los identificadores fiscales y el IBAN son valores de prueba para extraer como texto.